[CHAPTER] 5 | Lists and Dictionaries

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

A **list** allows the programmer to manipulate a sequence of data values of any types.
A **dictionary** organizes data values by association with other data values rather than by sequential position.

Lists

A list is a sequence of data values called **items** or **elements**.
Here are some real-world examples of lists:

- A shopping list for the grocery store
- A to-do list
- A roster for an athletic team
- A guest list for a wedding
- A recipe, which is a list of instructions
- A text document, which is a list of lines
- The names in a phone book

List Literals and Basic Operators

As you have seen, literal string values are written as sequences of characters enclosed in quote marks. In Python, a list literal is written as a sequence of data values separated by commas. The entire sequence is enclosed in square brackets ([and]). Here are some example list literals:

```
[1951, 1969, 1984]                      # A list of integers
["apples", "oranges", "cherries"]       # A list of strings
[]                                      # An empty list
```

You can also use other lists as elements in a list, thereby creating a list of lists. Here is one example of such a list: `[[5, 9], [541, 78]]`

```
>>> import math
>>> x = 2
>>> [x, math.sqrt(x)]
[2, 1.4142135623730951]
>>> [x + 1]
[3]
```

```
>>> first = [1, 2, 3, 4]
>>> second = list(range(1, 5))
>>> first
[1, 2, 3, 4]
>>> second
[1, 2, 3, 4]
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

The **list** function can build a list from any iterable sequence of elements, such as a string:

```
>>> third = list("Hi there!")
>>> third
['H', 'i', ' ' , 't', 'h', 'e', 'r', 'e', '!']
```

The function **len** and the subscript operator **[]** work just as they do for strings:

```
>>> len(first)
4
>>> first[0]
1
>>> first[2:4]
[3, 4]
```

Concatenation (+) and equality (==) also work as expected for lists:

```
>>> first + [5, 6]
[1, 2, 3, 4, 5, 6]
>>> first == second
True
```

```
>>> print("1234")
1234
>>> print([1, 2, 3, 4])
[1, 2, 3, 4]
```

The **print** function strips the quotation marks from a string, but it does not alter the look of a list:

for loop

```
>>> for number in [1, 2, 3, 4]:
        print(number, end = " ")
1 2 3 4
```

in operator

```
>>> 3 in [1, 2, 3]
True
>>> 0 in [1, 2, 3]
False
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Table 5-1 Some operators and functions used with lists

Operator or Function	What It Does
`L[<an integer expression>]`	Subscript used to access an element at the given index position.
`L[<start>:<end>]`	Slices for a sublist. Returns a new list.
`L1 + L2`	List concatenation. Returns a new list consisting of the elements of the two operands.
`print(L)`	Prints the literal representation of the list.
`len(L)`	Returns the number of elements in the list.
`list(range(<upper>))`	Returns a list containing the integers in the range **0** through **upper - 1**.
`==, !=, <, >, <=, >=`	Compares the elements at the corresponding positions in the operand lists. Returns **True** if all the results are true, or **False** otherwise.
`for <variable> in L: <statement>`	Iterates through the list, binding the variable to each element.
`<any value> in L`	Returns **True** if the value is in the list or **False** otherwise.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Replacing an Element in a List

The subscript operator is used to replace an element at a given position, as shown

```
>>> numbers = [2, 3, 4, 5]
>>> numbers
[2, 3, 4, 5]
>>> for index in range(len(numbers)):
        numbers[index] = numbers[index] ** 2
>>> numbers
[4, 9, 16, 25]
```

The first session shows how to replace each number in a list with its square:

```
>>> example = [1, 2, 3, 4]
>>> example
[1, 2, 3, 4]
>>> example[3] = 0
>>> example
[1, 2, 3, 0]
```

The next session uses the string method **split** to extract a list of the words in a sentence. These words are then converted to uppercase letters within the list:

```
>>> sentence = "This example has five words."
>>> words = sentence.split()
>>> words
['This', 'example', 'has', 'five', 'words.']
>>> for index in range(len(words)):
        words[index] = words[index].upper()
>>> words
['THIS', 'EXAMPLE', 'HAS', 'FIVE', 'WORDS.']
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

List Methods for Inserting and Removing Elements

Table 5-2 List methods for inserting and removing elements

List Method	What It Does
`L.append(element)`	Adds `element` to the end of `L`.
`L.extend(aList)`	Adds the elements of `aList` to the end of `L`.
`L.insert(index, element)`	Inserts `element` at `index` if `index` is less than the length of `L`. Otherwise, inserts `element` at the end of `L`.
`L.pop()`	Removes and returns the element at the end of `L`.
`L.pop(index)`	Removes and returns the element at `index`.

EIU
EASTERN INTERNATIONAL UNIVERSITY

The method **insert** expects an integer index and the new element as arguments. When the index is less than the length of the list, this method places the new element before the existing element at that index, after shifting elements to the right by one position. At the end of the operation, the new element occupies the given index position. When the index is greater than or equal to the length of the list, the new element is added to the end of the list. The next session shows **insert** in action:

```
>>> example = [1, 2]
>>> example
[1, 2]
>>> example.insert(1, 10)
>>> example
[1, 10, 2]
>>> example.insert(3, 25)
>>> example
[1, 10, 2, 25]
```

```
>>> example = [1, 2]
>>> example
[1, 2]
>>> example.append(3)
>>> example
[1, 2, 3]
>>> example.extend([11, 12, 13])
>>> example
[1, 2, 3, 11, 12, 13]
>>> example + [14, 15]
[1, 2, 3, 11, 12, 13, 14, 15]
>>> example
[1, 2, 3, 11, 12, 13]
```

The method **append** is a simplified version of **insert**. The method **append** expects just the new element as an argument and adds the new element to the end of the list. The method **extend** performs a similar operation, but adds the elements of its list argument to the end of the list.

The method **pop** is used to remove an element at a given position. If the position is not specified, **pop** removes and returns the last element. If the position is specified, **pop** removes the element at that position and returns it. In that case, the elements that followed the removed element are shifted one position to the left. The next session removes the last and first elements from the example list:

```
>>> example
[1, 2, 10, 11, 12, 13]
>>> example.pop()          # Remove the last element
13
>>> example
[1, 2, 10, 11, 12]
>>> example.pop(0)         # Remove the first element
1
>>> example
[2, 10, 11, 12]
```

Searching a List

```
aList = [34, 45, 67]
target = 45
if target in aList:
    print(aList.index(target))
else:
    print(-1)
```

Sorting a List

```
>>> example = [4, 2, 10, 8]
>>> example
[4, 2, 10, 8]
>>> example.sort()
>>> example
[2, 4, 8, 10]
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Mutator Methods and the Value None

Unfortunately, after the list object is sorted, this assignment has the result of setting the variable **aList** to the value **None**. The next **print** statement shows that the reference to the list object is lost:

```
>>> aList = aList.sort()
>>> print(aList)
None
```

Aliasing and Side Effects

In this example, a single list object is created and modified using the subscript operator. When the second element of the list named **first** is replaced, the second element of the list named **second** is replaced also. This type of change is what is known as a **side effect**. This happens because after the assignment **second = first**, the variables **first** and **second** refer to the exact same list object. They are **aliases** for the same object, as shown in Figure 5-1. This phenomenon is known as **aliasing**.

```
>>> first = [10, 20, 30]
>>> second = first
>>> first
[10, 20, 30]
>>> second
[10, 20, 30]
>>> first[1] = 99
>>> first
[10, 99, 30]
>>> second
[10, 99, 30]
```

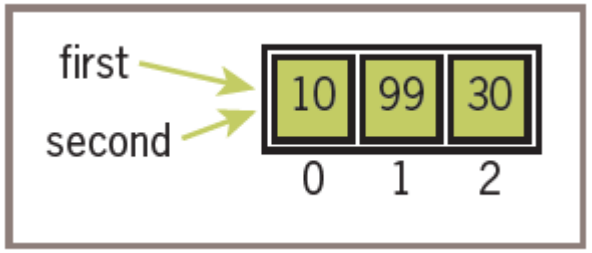


Figure 5-1 Two variables refer to the same list object

aliasing is not always a good thing when side effects are possible. Assignment creates an alias to the same object rather than a reference to a copy of the object. To prevent aliasing, you can create a new object and copy the contents of the original to it, as shown in the next session:

```
>>> third = []
>>> for element in first:
        third.append(element)
>>> first
[10, 99, 30]
>>> third
[10, 99, 30]
>>> first[1] = 100
>>> first
[10, 100, 30]
>>> third
[10, 99, 30]
```

The variables **first** and **third** refer to two different list objects, although their contents are initially the same, as shown in Figure 5-2. The important point is that they are not aliases, so you don't have to be concerned about side effects.

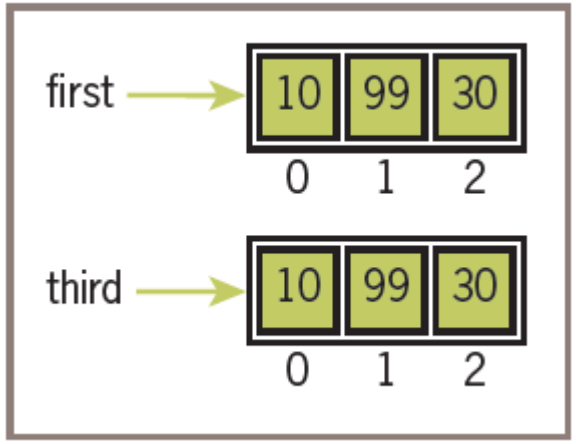


Figure 5-2 Two variables refer to different list objects

A simpler way to copy a list is to pass the source list to a call of the **list** function, as follows:

```
>>> third = list(first)
```

Equality: Object Identity and Structural Equivalence

Occasionally, programmers need to see whether two variables refer to the exact same object or to different objects. For example, you might want to determine whether one variable is an alias for another. The == operator returns **True** if the variables are aliases for the same object. Unfortunately, == also returns **True** if the contents of two different objects are the same. The first relation is called **object identity**, whereas the second relation is called **structural equivalence**. The == operator has no way of distinguishing between these two types of relations.

Python's **is** operator can be used to test for object identity. It returns **True** if the two operands refer to the exact same object, and it returns **False** if the operands refer to distinct objects (even if they are structurally equivalent). The next session shows the difference between == and **is**, and Figure 5-3 depicts the objects in question.

```
>>> first = [20, 30, 40]
>>> second = first
>>> third = list(first)
          # Or first[:]

>>> first == second
True
>>> first == third
True
>>> first is second
True
>>> first is third
False
```

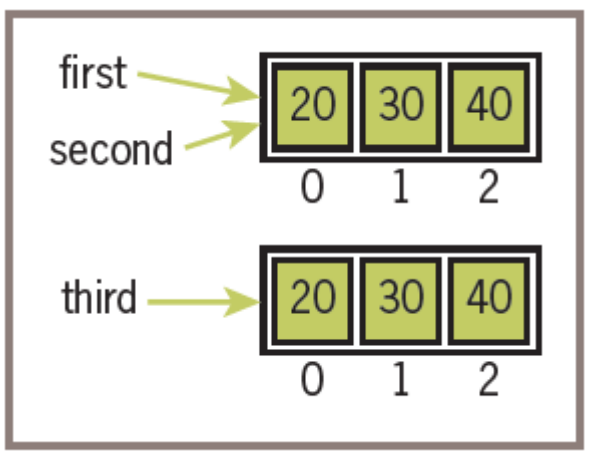


Figure 5-3 Three variables and two distinct list objects

Example: Using a List to Find the Median of a Set of Numbers

When run with an input file whose contents are

```
3 2 7
8 2 1
5
```

the script produces the following output:

```
The median is 3.0
```

```
"""
File: median.py
Prints the median of a set of numbers in a file.
"""

fileName = input("Enter the filename: ")
f = open(fileName, 'r')

# Input the text, convert it to numbers, and
# add the numbers to a list
numbers = []
for line in f:
    words = line.split()
    for word in words:
        numbers.append(float(word))

# Sort the list and print the number at its midpoint
numbers.sort()
midpoint = len(numbers) // 2
print("The median is", end = " ")
if len(numbers) % 2 == 1:
    print(numbers[midpoint])
else:
    print((numbers[midpoint] + numbers[midpoint - 1]) / 2)
```

Tuples

A **tuple** is a type of sequence that resembles a list, except that, unlike a list, a tuple is immutable. You indicate a tuple literal in Python by enclosing its elements in parentheses instead of square brackets. The next session shows how to create several tuples:

```
>>> fruits = ("apple", "banana")
>>> fruits
('apple', 'banana')
>>> meats = ("fish", "poultry")
>>> meats
('fish', 'poultry')
>>> food = meats + fruits
>>> food
('fish', 'poultry', 'apple', 'banana')
>>> veggies = ["celery", "beans"]
>>> tuple(veggies)
('celery', 'beans')
```

You must be careful when using a tuple of one element. When that is the case, you place a comma after the expression within the parentheses, as shown in the following session:

```
>>> badSingleton = (3)
>>> badSingleton
3
>>> goodSingleton = (3,)
>>> goodSingleton
(3,)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Assume that the variable **data** refers to the list **[5, 3, 7]**. Write the values of the following expressions:

   a. `data[2]`
   b. `data[-1]`
   c. `len(data)`
   d. `data[0:2]`
   e. `0 in data`
   f. `data + [2, 10, 5]`
   g. `tuple(data)`

2. Assume that the variable data refers to the list **[5, 3, 7]**. Write the expressions that perform the following tasks:

   a. Replace the value at position 0 in **data** with that value's negation.
   b. Add the value 10 to the end of **data**.
   c. Insert the value 22 at position 2 in **data**.
   d. Remove the value at position 1 in **data**.
   e. Add the values in the list **newData** to the end of **data**.
   f. Locate the index of the value 7 in **data**, safely.
   g. Sort the values in **data**.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Defining Simple Functions

The Syntax of Simple Function Definitions

The definition of this function consists of a **header** and a **body**. Here is the code:

```
def square(x):
    """Returns the square of x."""
    return x * x
```

```
>>> square(2)
4
>>> square(6)
36
>>> square(2.5)
6.25
```

The header includes the keyword **def** as well as the function name and list of parameters. The function's body contains one or more statements. Here is the syntax:

```
def <function name>(<parameter-1>, ..., <parameter-n>):
    <body>
```

Our next example function computes the average value in a list of numbers. The function might be used as follows:

```
>>> average([1, 3, 5, 7])
4.0
```

```
def average(lyst):
    """Returns the average of the numbers in lyst."""
    theSum = 0
    for number in lyst:
        theSum += number
    return theSum / len(lyst)
```

Defining a `main` Function

In scripts that include the definitions of several cooperating functions, it is often useful to define a special function named **`main`** that serves as the entry point for the script. This function usually expects no arguments and returns no value. Its purpose might be to take inputs, process them by calling other functions, and print the results. The definition of the **`main`** function and the other function definitions need appear in no particular order in the script, as long as **`main`** is called at the very end of the script. The next example shows a complete script that is organized in the manner just described. The **`main`** function prompts the user for a number, calls the **`square`** function to compute its square, and prints the result.

```
"""
File: computesquare.py
Illustrates the definition of a main function.
"""

def main():
    """The main function for this script."""
    number = float(input("Enter a number: "))
    result = square(number)
    print("The square of", number, "is", result)

def square(x):
    """Returns the square of x."""
    return x * x

# The entry point for program execution
if __name__ == "__main:"__
    main()
```

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Dictionaries

A dictionary organizes information by **association**, not position. For example, when you use a dictionary to look up the definition of "mammal," you don't start at page 1; instead, you turn directly to the words beginning with "M." Phone books, address books, encyclopedias, and other reference sources also organize information by association. In computer science, data structures organized by association are also called **tables** or **association lists**. In Python, a **dictionary** associates a set of **keys** with **values**. For example, the keys in *Webster's Dictionary* comprise the set of words, whereas the associated data values are their definitions. In this section, we examine the use of dictionaries in data processing.

Dictionary Literals

A Python dictionary is written as a sequence of key/value pairs separated by commas. These pairs are sometimes called **entries**. The entire sequence of entries is enclosed in curly braces (`{` and `}`). A colon (`:`) separates a key and its value.

```
A phone book: {"Savannah":"476-3321", "Nathaniel":"351-7743"}
```

Personal information: `{"Name":"Molly", "Age":18}`

Here is an example of an empty dictionary: `{}`

EIU
EASTERN INTERNATIONAL UNIVERSITY

Adding Keys and Replacing Values

You add a new key/value pair to a dictionary by using the subscript operator `[]`. The form of this operation is the following:

```
<a dictionary>[<a key>] = <a value>
```

The next code segment creates an empty dictionary and adds two new entries:

```
>>> info = {}
>>> info["name"] = "Sandy"
>>> info["occupation"] = "hacker"
>>> info
{'name':'Sandy', 'occupation':'hacker'}
```

The subscript is also used to replace a value at an existing key, as follows:

```
>>> info["occupation"] = "manager"
>>> info
{'name':'Sandy', 'occupation':'manager'}
```

Here is a case of the same operation used for two different purposes: insertion of a new entry and modification of an existing entry. As a rule, when the key is absent the dictionary, and its value are inserted; when the key already exists, its associated value is replaced.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Accessing Values

```
>>> info["name"]
'Sandy'
>>> info["job"]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    info["job"]
KeyError: 'job'
```

If the existence of a key is uncertain, the programmer can test for it using the operator **in**, as follows:

```
>>> if "job" in info:
        print(info.["job"])
```

```
>>> print(info.get("job", None))
None
```

Removing Keys

To delete an entry from a dictionary, one removes its key using the method **pop**. This method expects a key and an optional default value as arguments. If the key is in the dictionary, it is removed, and its associated value is returned. Otherwise, the default value is returned. If **pop** is used with just one argument, and this key is absent from the dictionary, Python raises an exception.

```
>>> print(info.pop("job", None))
None
>>> print(info.pop("occupation"))
manager
>>> info
{'name':'Sandy'}
```

Traversing a Dictionary

When a **for** loop is used with a dictionary, the loop's variable is bound to each key in an unspecified order. The next code segment prints all of the keys and their values in our **info** dictionary:

```
for key in info:
    print(key, info[key])
```

```
>>> grades = {90:'A', 80:'B', 70:'C'}
>>> list(grades.items())
[(80,'B'), (90,'A'), (70,'C')]

for (key, value) in grades.items():
   print(key, value)
```

Alternatively, you could use the dictionary method **items()** to access the dictionary's entries. The next session shows a run of this method with a dictionary of grades:

Note that the entries are represented as tuples within the list. A tuple of variables can then access the key and value of each entry in this list within a **for** loop:

The use of a tuple of variables rather than a simple variable in the **for** loop is a powerful way to implement this traversal. On each pass through the loop, the variables **key** and **value** within the tuple are assigned the key and value of the current entry in the list. The use of a structure containing variables to access data within another structure is called **pattern matching**.

EIU
EASTERN INTERNATIONAL UNIVERSITY

If a special ordering of the keys is needed, you can obtain a list of keys using the **keys** method and process this list to rearrange the keys. For example, you can sort the list and then traverse it to print the entries of the dictionary in alphabetical order:

```
theKeys = list(info.keys())
theKeys.sort()
for key in theKeys:
    print(key, info[key])
```

Table 5-4 Some commonly used dictionary operations

Dictionary Operation	What It Does
`len(d)`	Returns the number of entries in **d**.
`d[key]`	Used for inserting a new key, replacing a value, or obtaining a value at an existing key.
`d.get(key [, default])`	Returns the value if the key exists or returns the default if the key does not exist. Raises an error if the default is omitted and the key does not exist.
`d.pop(key [, default])`	Removes the key and returns the value if the key exists or returns the default if the key does not exist. Raises an error if the default is omitted and the key does not exist.
`list(d.keys())`	Returns a list of the keys.
`list(d.values())`	Returns a list of the values.
`list(d.items())`	Returns a list of tuples containing the keys and values for each entry.
`d.clear()`	Removes all the keys.
`for key in d:`	**key** is bound to each key in **d** in an unspecified order.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Example: The Hexadecimal System Revisited

Here is the definition of the lookup table required for hex-to-binary conversions:

```
hexToBinaryTable = {'0':'0000', '1':'0001', '2':'0010',
                    '3':'0011', '4':'0100', '5':'0101',
                    '6':'0110', '7':'0111', '8':'1000',
                    '9':'1001', 'A':'1010', 'B':'1011',
                    'C':'1100', 'D':'1101', 'E':'1110',
                    'F':'1111'}
```

The function itself, named **convert**, is simple. It expects two parameters: a string representing the number to be converted and a table of associations of digits. Here is the code for the function, followed by a sample session:

```
def convert(number, table):
    """Builds and returns the base two representation of
    number."""
    binary = ""
    for digit in number:
        binary = table[digit] + binary
    return binary

>>> convert("35A", hexToBinaryTable)
'001101011010'
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Example: Finding the Mode of a List of Values

```
fileName = input("Enter the filename: ")
f = open(fileName, 'r')

# Input the text, convert its words to uppercase, and
# add the words to a list
words = []
for line in f:
    for word in line.split():
        words.append(word.upper())
# Obtain the set of unique words and their
# frequencies, saving these associations in
# a dictionary
theDictionary = {}
for word in words:
    number = theDictionary.get(word, None)
    if number == None:
        # word entered for the first time
        theDictionary[word] = 1
  else:
      # word already seen, increment its number
      theDictionary[word] = number + 1

# Find the mode by obtaining the maximum value
# in the dictionary and determining its key
theMaximum = max(theDictionary.values())
for key in theDictionary:
    if theDictionary[key] == theMaximum:
        print("The mode is", key)
        break
```

Exercises

1. Assume that the variable **data** refers to the dictionary **{'b':20, 'a':35}**. Write the values of the following expressions:

   a. `data['a']`

   b. `data.get('c', None)`

   c. `len(data)`

   d. `data.keys()`

   e. `data.values()`

   f. `data.pop('b')`

   g. `data # After the pop above`

2. Assume that the variable **data** refers to the dictionary **{'b':20, 'a':35}**. Write the expressions that perform the following tasks:

   a. Replace the value at the key **'b'** in **data** with that value's negation.

   b. Add the key/value pair **'c':40** to **data**.

   c. Remove the value at key **'b'** in **data**, safely.

   d. Print the keys in **data** in alphabetical order.

Projects



1. A group of statisticians at a local college has asked you to create a set of functions that compute the median and mode of a set of numbers, as defined in Section 5.4. Define these functions in a module named **stats.py**. Also include a function named **`mean`**, which computes the average of a set of numbers. Each function should expect a list of numbers as an argument and return a single number. Each function should return 0 if the list is empty. Include a **`main`** function that tests the three statistical functions with a given list.

2. Write a program that allows the user to navigate the lines of text in a file. The program should prompt the user for a filename and input the lines of text into a list. The program then enters a loop in which it prints the number of lines in the file and prompts the user for a line number. Actual line numbers range from 1 to the number of lines in the file. If the input is 0, the program quits. Otherwise, the program prints the line associated with that number.

3. Write a program that inputs a text file. The program should print the unique words in the file in alphabetical order.

EIU
EASTERN INTERNATIONAL UNIVERSITY

[CHAPTER] 6

Design with Functions

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Functions Eliminate Redundancy

The first way that functions serve as abstraction mechanisms is by eliminating redundant, or repetitious, code. To explore the concept of redundancy, let's look at a function named **summation**, which returns the sum of the numbers within a given range of numbers.

```
def summation(lower, upper):
    """Arguments: A lower bound and an upper bound
    Returns: the sum of the numbers from lower through
    upper
    """
    result = 0
    while lower <= upper:
        result += lower
        lower += 1
    return result
```

```
>>> summation(1,4)      # The summation of the numbers 1..4
10
>>> summation(50,100)   # The summation of the numbers 50..100
3825
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Defining a Recursive Function

A recursive function is a function that calls itself. To prevent a function from repeating itself indefinitely, it must contain at least one selection statement. This statement examines a condition called a **base case** to determine whether to stop or to continue with another **recursive step**.

Let's examine how to convert an iterative algorithm to a recursive function. Here is a definition of a function **displayRange** that prints the numbers from a lower bound to an upper bound:

```
def displayRange(lower, upper):
    """Outputs the numbers from lower through upper."""
    while lower <= upper:
        print(lower)
        lower = lower + 1
```

The equivalent recursive function performs similar primitive operations, but the loop is replaced with a selection statement, and the assignment statement is replaced with a **recursive call** of the function.

```
def displayRange(lower, upper):
    """Outputs the numbers from lower through upper."""
    if lower <= upper:
        print(lower)
        displayRange(lower + 1, upper)
```

Our next example is a recursive function that builds and returns a value.

```
def summation(lower, upper):
    """Returns the sum of the numbers from lower through
    upper."""
    if lower > upper:
        return 0
    else:
        return lower + summation (lower + 1, upper)
```

The recursive call of **summation** adds up the numbers from **lower + 1** through **upper**. The function then adds **lower** to this result and returns it.

Tracing a Recursive Function

```
>>> summation (1, 4, 0)
1 4
    2 4
        3 4
            4 4
                5 4
                0
            4
        7
    9
10
10
```

```
def summation(lower, upper, margin):
    """Returns the sum of the numbers from lower through
upper,
    and outputs a trace of the arguments and return values
    on each call"""
    blanks = " " * margin
    print(blanks, lower, upper)
    if lower > upper:
        print(blanks, 0)
        return 0
    else:
        result = lower + summation(lower + 1, upper,
                                   margin + 4)
        print(blanks, result)
        return result
```

Using Recursive Definitions to Construct Recursive Functions

Recursive functions are frequently used to design algorithms for computing values that have a **recursive definition**. A recursive definition consists of equations that state what a value is for one or more base cases and one or more recursive cases. For example, the Fibonacci sequence is a series of values with a recursive definition. The first and second numbers in the Fibonacci sequence are 1. Thereafter, each number in the sequence is the sum of its two predecessors, as follows:

1 1 2 3 5 8 13 . . .

More formally, a recursive definition of the *n*th **Fibonacci number** is the following:

```
Fib(n) = 1, when n = 1 or n = 2
Fib(n) = Fib(n - 1) + Fib(n - 2), for all n > 2
```

Given this definition, you can construct a recursive function that computes and returns the *n*th Fibonacci number. Here it is:

```
def fib(n):
    """Returns the nth Fibonacci number."""
    if n < 3:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
```

Note that the base case as well as the two recursive steps return values to the caller.

Recursion in Sentence Structure

the **nounPhrase** function would call the **prepositionalPhrase** function, which in turn calls **nounPhrase** again. This phenomenon is known as **indirect recursion**. To keep this process from going on forever, **nounPhrase** must also have the option to not generate a prepositional phrase. Here is a statement of the modified rule, which expresses an optional phrase within the square brackets:

Nounphrase = Article Noun [Prepositionalphrase]

```
def nounPhrase():
    """Returns a noun phrase, which is an article followed
    by a noun, and an optional prepositional phrase."""
    phrase = random.choice(articles) + " " + random.choice(nouns)
    prob = random.randint(1, 4)
    if prob == 1:
        return phrase + " " + prepositionalPhrase()
    else:
        return phrase

def prepositionalPhrase():
    """Builds and returns a prepositional phrase."""
    return random.choice(prepositions) + " " + nounPhrase()
```

The code for a revised **nounPhrase** function generates a modifying prepositional phrase approximately 25% of the time:

Infinite Recursion

execution with a message indicating a **stack overflow error**.

```
>>> def runForever(n):
        if n > 0:
            runForever(n)
        else:
            runForever(n - 1)
```

```
>>> runForever(1)
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    runForever(1)
  File "<pyshell#5>", line 3, in runForever
    runForever(n)
  File "<pyshell#5>", line 3, in runForever
    runForever(n)
  File "<pyshell#5>", line 3, in runForever
    runForever(n)
  [Previous line repeated 989 more times]
  File "<pyshell#5>", line 2, in runForever
    if n > 0:
RecursionError: maximum recursion depth exceeded in comparison
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

The Costs and Benefits of Recursion

Although recursive solutions are often more natural and elegant than their iterative counterparts, they come with a cost. The run-time system on a real computer, such as the PVM, must devote some overhead to recursive function calls. At program startup, the PVM reserves an area of memory named a **call stack**. For each call of a function, recursive or otherwise, the PVM must allocate on the call stack a small chunk of memory called a **stack frame**. In this type of storage, the system places the values of the arguments and the return address for each function call. Space for the function call's return value is also reserved in its stack frame. When a call returns or completes its execution, the return address is used to locate the next instruction in the caller's code, and the memory for the stack frame is deallocated. The stack frames for the process generated by `displayRange(1, 3)` are shown in Figure 6-4. The frames in the figure include storage for the function's arguments only.

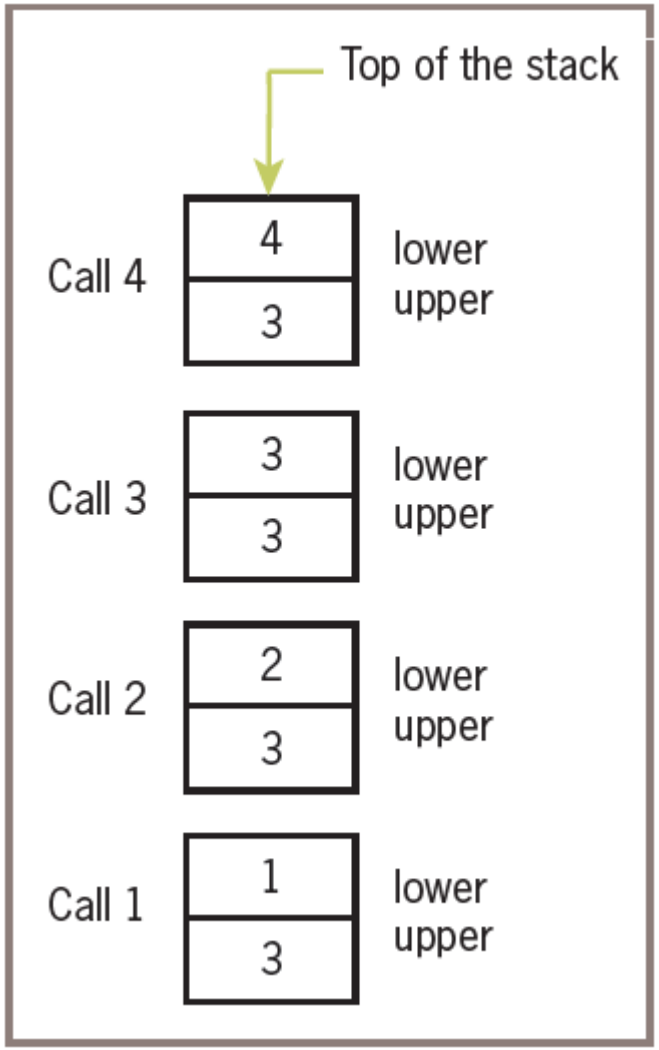


Figure 6-4 The stack frames for `displayRange(1, 3)`

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. The factorial of a positive integer **n**, **fact(n)**, is defined recursively as follows:

   $fact(n) = 1$, when $n = 1$

   $fact(n) = n * fact(n - 1)$, otherwise

   Define a recursive function **fact** that returns the factorial of a given positive integer.

2. Explain what happens when the following recursive function is called with the value 4 as an argument:

```
def example(n):
    if n > 0:
        print(n)
        example(n - 1)
```

3. Explain what happens when the following recursive function is called with the value 4 as an argument:

```
def example(n):
    if n > 0:
        print(n)
        example(n)
    else:
        example(n - 1)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



4. Explain what happens when the following recursive function is called with the values **"hello"** and 0 as arguments:

```
def example(aString, index):
    if index < len(aString):
        example(aString, index + 1)
        print(aString[index], end = "")
```

5. Explain what happens when the following recursive function is called with the values **"hello"** and 0 as arguments:

```
def example(aString, index):
    if index == len(aString):
        return ""
    else:
        return aString[index] + example(aString, index + 1)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Using Keywords for Default and Optional Arguments

The programmer can also specify **optional arguments** with default values in any function definition. Here is the syntax:

```
def <function name>(<required arguments>,
                    <key-1> = <val-1>, ... <key-n> = <val-n>)
```

```
def repToInt(repString, base):
    """Converts the repString to an int in the base
    and returns this int."""
    decimal = 0
    exponent = len(repString) - 1
    for digit in repString:
        decimal = decimal + int(digit) * base ** exponent
        exponent -= 1
    return decimal
```

```
def repToInt(repString, base = 2):

>>> repToInt("10", 10)
10
>>> repToInt("10", 8)
  # Override the default to here
8
>>> repToInt("10", 2)
  # Same as the default, not necessary
2
>>> repToInt("10")
  # Base 2 by default
2
```

Functions as First-Class Data Objects

In Python, functions can be treated as **first-class data objects**. This means that they can be assigned to variables (as they are when they are defined), passed as arguments to other functions, returned as the values of other functions, and stored in data structures such as lists and dictionaries. The next session shows some of the simpler possibilities:

```
>>> def example(functionArg, dataArg):
        return functionArg(dataArg)
>>> example(abs, -4)
4
>>> example(math.sqrt, 2)
1.4142135623730951
```

```
>>> abs                              # See what abs looks like
<built-in function abs>
>>> import math
>>> math.sqrt
<built-in function sqrt>
>>> f = abs                          # f is an alias for abs
>>> f                                # Evaluate f
<built-in function abs>
>>> f(-4)                           # Apply f to an argument
4
>>> funcs = [abs, math.sqrt]        # Put the functions in a list
>>> funcs
[<built-in function abs>,   <built-in function sqrt>]
>>> funcs[1](2)                     # Apply math.sqrt to 2
1.4142135623730951
```

Mapping

The first type of useful higher-order function to consider is called a **mapping**.

```
>>> words = ["231", "20", "-45", "99"]
>>> map(int, words)          # Convert all strings to ints
<map object at 0x14cbd90>
>>> words                    # Original list is not changed
['231', '20', '-45', '99']
>>> words = list(map(int, words))    # Reset variable to change it
>>> words
[231, 20, -45, 99]
```

```
def changePerson(sentence):
    """Replaces first person pronouns with second person pronouns."""
    words = sentence.split( )
    replyWords = []
    for word in words:
        replyWords.append(replacements.get(word, word))
    return " ".join(replyWords)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
def changePerson(sentence):
    """Replaces first person pronouns with second person pronouns."""

    def getWord(word):
        return replacements.get(word, word)

    return " ".join(map(getWord, sentence.split()))
```

Filtering

A second type of higher-order function is called a **filtering**. In this process, a function called a **predicate** is applied to each value in a list. If the predicate returns **True**, the value passes the test and is added to a filter object (similar to a map object). Otherwise, the value is dropped from consideration.

```
>>> def odd(n): return n % 2 == 1
>>> list(filter(odd, range(10)))
[1, 3, 5, 7, 9]
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Reducing

Our final example of a higher-order function is called a **reducing**. Here we take a list of values and repeatedly apply a function to accumulate a single data value. A summation is a good example of this process. The first value is added to the second value, then the sum is added to the third value, and so on, until the sum of all the values is produced.

```
>>> from functools import reduce
>>> def add(x, y): return x + y
>>> def multiply(x, y): return x * y
>>> data = [1, 2, 3, 4]
>>> reduce(add, data)
10
>>> reduce(multiply, data)
24
```

Using lambda to Create Anonymous Functions

Python includes a mechanism called **lambda** that allows the programmer to create functions in this manner. A **lambda** is an **anonymous function**. It has no name of its own, but it contains the names of its arguments as well as a single expression. When the **lambda** is applied to its arguments, its expression is evaluated, and its value is returned.

EIU
EASTERN INTERNATIONAL UNIVERSITY

The syntax of a **lambda** is very tight and restrictive: `lambda <argname-1, ..., argname-n>: <expression>`

```
>>> data = [1, 2, 3, 4]
>>> reduce(lambda x, y: x + y, data)     # Produce the sum
10
>>> reduce(lambda x, y: x * y, data)     # Produce the product
24
```

```
def summation(lower, upper):
    """Returns the sum of the numbers from lower
    through upper."""
    if lower > upper:
        return 0
    else:
        return reduce(lambda x, y: x + y,
                      range(lower, upper + 1))
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Creating Jump Tables

A **jump table** is a dictionary of functions keyed by command names. At program startup, the functions are defined and then the jump table is loaded with the command names and their associated functions. The function **runCommand** uses its **command** argument to look up the function in the jump table and then calls this function. Here is the modified version of **runCommand**:

```
def runCommand(command):     # How simple can it get?
    jumpTable[command]()
```

Note that this function makes two important simplifying assumptions: the command string is a key in the jump table, and its associated function expects no arguments.

Let's assume that the functions **insert**, **replace**, and **remove** are keyed to the commands **'1'**, **'2'**, and **'3'**, respectively. Then the setup of the jump table is straightforward:

```
# The functions named insert, replace, and remove
# are defined earlier
jumpTable = {}
jumpTable['1'] = insert
jumpTable['2'] = replace
jumpTable['3'] = remove
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Write the code for a mapping that generates a list of the absolute values of the numbers in a list named **`numbers`**.
2. Write the code for a filtering that generates a list of the positive numbers in a list named **`numbers`**. You should use a **`lambda`** to create the auxiliary function.
3. Write the code for a reducing that creates a single string from a list of strings named **`words`**.
4. Write a recursive function that expects a pathname as an argument. The pathname can be either the name of a file or the name of a directory. If the pathname refers to a file, its name is displayed, followed by its contents. Otherwise, if the pathname refers to a directory, the function is applied to each name in the directory. Test this function in a new program.
5. Write a program that computes and prints the average of the numbers in a text file. You should make use of two higher-order functions to simplify the design.

EIU
EASTERN INTERNATIONAL UNIVERSITY